



Institut Teknologi Bandung

Directorate of Sustainable Professional Education

CHAPTER 20

Conclusions

Everything this course covered, arranged so you can reconstruct the decisions rather than remember the details — and where to go next.

Duration 2 hours (1 session)

Instructor Prof. Bambang Riyanto Trilaksono · Rahman Indra Kesuma, S.Kom., M.Cs.

Source Chollet & Watson, Deep Learning with Python 3e — chapter 20

Session Objectives

By the end of this session, participants can:

- 1 Place **AI, machine learning, deep learning, and generative AI** correctly inside one another.
- 2 State in one paragraph **what a deep learning model is** and how learning happens.
- 3 Name the **four enabling factors** behind the current revolution, and why no single one of them explains it.
- 4 Run the **universal workflow** from problem definition to deployment, and say what each step protects against.
- 5 Pick the right **architecture family** for a given pair of input and output modalities.
- 6 Write the correct **final layer, activation, and loss** for each of the five standard task types from memory.
- 7 State the **three structural limitations** honestly to a non-technical stakeholder.
- 8 Keep learning after this course: **Kaggle, arXiv, and the Keras ecosystem**.

BOOK

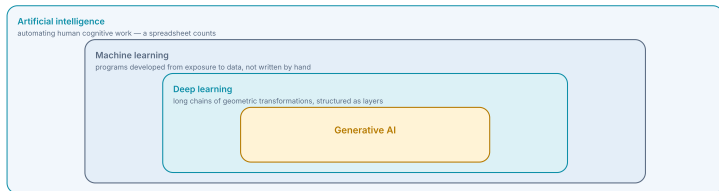
[Chapter 20 — full text](#)

01

Key concepts in review

If you ever need a quick refresher, this is the section to reread.

Four terms, nested inside one another



Every ring is a strict subset of the one around it. A chain of four boxes says something else entirely.

Each term is a proper subset of the one around it. Step outward-in and watch the narrowing.

Said precisely

TERM	DEFINITION
AI	An ancient, broad field: all attempts to automate human cognitive processes — from an Excel spreadsheet to a humanoid robot.
Machine learning	Automatically developing programs (models) purely from exposure to training data. Took off in the 1990s, dominant by the 2000s.
Deep learning	Models that are long chains of geometric transformations , structured into layers parameterized by weights.
Generative AI	Deep learning models that generate text, images, video, or sound — very large, trained self-supervised to reconstruct corrupted input.

The self-supervised objective is what lets generative models learn sophisticated **maps of their input space** — embedding manifolds that can then be sampled from.

Why deep learning is not on equal footing with the alternatives

Given sufficient training data, appropriately labelled, deep learning can extract from perceptual data **almost anything a human could**. It is sometimes said to have *solved perception* — true only for a fairly narrow definition of perception.

This singlehandedly brought about the **third and by far the largest AI summer**: a period of intense interest, investment, and hype. We are in the middle of it.

The hype may recede; the impact will not

*Deep learning could be analogous to **the internet**: overly hyped for a few years, but in the longer term still a major revolution that will transform our economy and our lives.*

Section 20.1.2

One reason for optimism: **even with no further technological progress in the next decade, deploying existing algorithms to every applicable problem would be a game changer for most industries.** Short-term expectations are overoptimistic; the full deployment will likely take decades.

The most surprising thing is how simple it is

"It's not complicated, it's just a lot of it."

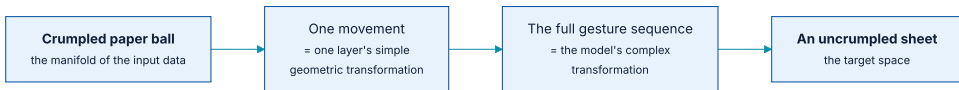
Richard Feynman, on the universe — quoted in section 20.1.3



The knowledge of a model is stored in its weights; learning is finding good values for them.

In deep learning, everything is a vector

That transformation must be **differentiable**, which is what lets gradient descent learn its parameters. Intuitively: the morphing from inputs to outputs must be **smooth and continuous** — a significant constraint, and the source of every limitation in chapter 19.



Deep learning models are mathematical machines for uncrumpling complicated manifolds of high-dimensional data.

Meaning as geometry

The whole thing hinges on two ideas: that **meaning derives from the pairwise relationships between things** — words in a language, pixels in an image — and that **those relationships can be captured by a distance function**.

Whether the brain implements meaning via geometric spaces is an entirely separate question. Vector spaces are computationally efficient, but other data structures for intelligence are easily envisioned — **graphs** in particular.

Neural networks originally emerged from the idea of using graphs to encode meaning; the surrounding field was called **connectionism**. That is the only reason for the name today.

They are neither neural nor networks

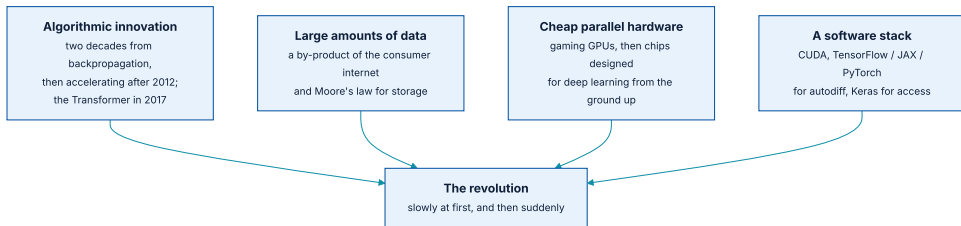
Neural network exists **purely for historical reasons**. It is an extremely misleading name — these are neither neural nor networks, and they have **hardly anything to do with the brain**.

More appropriate names, all of which emphasise that continuous geometric space manipulation is at the core:

- ♦ **Layered representations learning**
- ♦ **Hierarchical representations learning**
- ♦ **Deep differentiable models**
- ♦ **Chained geometric transforms**

This is not pedantry. The brain metaphor is precisely what invites the anthropomorphizing that chapter 19 warned against.

Four enabling factors, no single breakthrough



Remove any one of the four and the last decade does not happen.

Where the data and the hardware came from

The data

A by-product of the **consumer internet** and Moore's law applied to storage media. Today, state-of-the-art language models are trained on **a large fraction of the entire internet**.

The hardware

First **gaming GPUs**, then chips designed from the ground up for deep learning. NVIDIA's CEO took note of the boom early and bet the company's future on it.

And on top of both, a stack of software: **CUDA**, then **TensorFlow, JAX, and PyTorch** doing automatic differentiation, then **Keras** making the whole thing accessible to most people.

A tool in every developer's toolbox

In the future, deep learning will not be used only by researchers and engineers with an academic profile. **It will be a tool in the toolbox of every developer, much like web technology today.**

Everyone needs to build intelligent apps. Just as every business today needs a website, **every product will need to intelligently make sense of user-generated data.**

Bringing that about requires tools that make deep learning radically easy to use and accessible to anyone with basic coding ability. **Keras has been the first major step in that direction**, and it is why this course is taught in it.

02

The universal workflow

The hard part is everything before the model, and everything after it.

Keras cannot help you with the hard part

Understanding the problem domain well enough to determine **what to predict, from what data, and how to measure success** is a prerequisite for any successful application. **It is not something advanced tools can help you with.**

The workflow, end to end



Seven steps, of which only one is about the model.

What each step protects against

STEP	THE FAILURE IT PREVENTS
Define the problem	Building a model for a prediction nobody needs, or that the data cannot support.
Choose a measure of success	Optimizing a convenient number instead of a useful one — the shortcut rule from chapter 19.
Prepare the validation process	Label leakage. With temporal prediction, validation and test data must be <i>posterior</i> to the training data.
Beat a common-sense baseline	Spending months on a problem where machine learning does not work at all.
Overfit, then regularize	Regularizing a model that was never big enough to fit the data in the first place.
Keep a separate test set	Validation-set overfitting from hyperparameter tuning. This is the entire purpose of the third split.

Deployment is part of the workflow, not after it

Then **keep monitoring its performance on real-world data**, and use those findings to refine the next iteration.

The arrow from deployment back to problem definition is the one that turns a project into a system. Chapter 19's warning about distribution shift is not a research concern: **production data drifts away from your training set from the day you ship.**

03

Key network architectures

An architecture encodes assumptions about the structure of the data.

An architecture is a hypothesis space

A network architecture **encodes assumptions about the structure of the data** — a hypothesis space within which the search for a good model proceeds.

Whether an architecture works on a problem depends **entirely** on the match between the structure of the data and the assumptions of the architecture. That is the single question to ask when choosing one.

These types combine easily into larger multimodal models. **Deep learning layers are LEGO bricks for information processing.**

Input, output, architecture

INPUT	OUTPUT	MODEL
Vector data	Class probability, regression value	Densely connected network
Timeseries data	Class probability, regression value	RNN, Transformer
Images	Class probability, regression value	ConvNet
Text	Class probability, regression value	Transformer
Text, images	Text	Transformer
Text, images	Images	VAE, diffusion model

Six rows covering the whole course. If you remember nothing else from this deck, **remember this table** — it turns a vague problem statement into a starting architecture.

No assumed structure at all

Contrast a 2D convolution layer, which only looks at **local** relationships. The absence of assumptions is both the strength and the weakness.

They are also the **final classification or regression stage** of most networks: the ConvNets of chapter 8 and the recurrent networks of chapter 13 all end in one or two `Dense` layers.

Last layer, activation, and loss — from memory

TASK	LAST LAYER	LOSS
Binary classification	<code>Dense(1, activation="sigmoid")</code>	<code>binary_crossentropy</code>
Single-label, multiclass	<code>Dense(n, activation="softmax")</code>	<code>categorical_crossentropy</code> (one-hot targets)
Single-label, integer targets	<code>Dense(n, activation="softmax")</code>	<code>sparse_categorical_crossentropy</code>
Multilabel, multiclass	<code>Dense(n, activation="sigmoid")</code>	<code>binary_crossentropy</code> (k-hot targets)
Regression	<code>Dense(n)</code> — no activation	<code>mean_squared_error</code>

The multilabel row is the one people get wrong. **Sigmoid, not softmax** — softmax forces the outputs to sum to one, which is exactly what you do not want when a sample can have several classes.

The same three lines, three different endings

binary classification

PYTHON

```
inputs = keras.Input(shape=(num_input_features,))
x = layers.Dense(32, activation="relu")(inputs)
x = layers.Dense(32, activation="relu")(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs, outputs)
model.compile(optimizer="rmsprop", loss="binary_crossentropy")
```

single-label multiclass

PYTHON

```
outputs = layers.Dense(num_classes, activation="softmax")(x)
model = keras.Model(inputs, outputs)
model.compile(optimizer="rmsprop", loss="categorical_crossentropy")
```

Targets should be **0 or 1** for the first, **one-hot** for the second — or integers, with `sparse_categorical_crossentropy`.

A sequence-to-sequence Transformer, from KerasHub layers

PYTHON

```
from keras_hub.layers import TokenAndPositionEmbedding
from keras_hub.layers import TransformerDecoder, TransformerEncoder

encoder_inputs = keras.Input(shape=(src_seq_length,), dtype="int64")
x = TokenAndPositionEmbedding(vocab_size, src_seq_length, embed_dim)(
    encoder_inputs
)
encoder_outputs = TransformerEncoder(intermediate_dim, num_heads)(x)

decoder_inputs = keras.Input(shape=(tgt_seq_length,), dtype="int64")
x = TokenAndPositionEmbedding(vocab_size, tgt_seq_length, embed_dim)(
    decoder_inputs
)
x = TransformerDecoder(intermediate_dim, num_heads)(x, encoder_outputs)
decoder_outputs = layers.Dense(vocab_size, activation="softmax")(x)

transformer = keras.Model([encoder_inputs, decoder_inputs], decoder_outputs)
```

`TokenAndPositionEmbedding` is the tied token-plus-position layer from chapters 15 and 16. **If you are processing a single sequence, delete the decoder half** and keep the encoder.

The same transformation, applied everywhere

The idea applies to any dimensionality: `Conv1D` for continuous sequences, `Conv2D` for images, `Conv3D` for volumes. `SeparableConv2D` is a leaner, more efficient alternative.

ConvNets are stacks of convolution and pooling layers. Pooling **downsamples spatially** — required to keep feature maps a reasonable size as feature counts grow, and to let later convolutions see a greater spatial extent.

A typical image classifier, and what to add when it goes deep

PYTHON

```
inputs = keras.Input(shape=(height, width, channels))
x = layers.SeparableConv2D(32, 3, activation="relu")(inputs)
x = layers.SeparableConv2D(64, 3, activation="relu")(x)
x = layers.MaxPooling2D(2)(x)
x = layers.SeparableConv2D(64, 3, activation="relu")(x)
x = layers.SeparableConv2D(128, 3, activation="relu")(x)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(32, activation="relu")(x)
outputs = layers.Dense(num_classes, activation="softmax")(x)
model = keras.Model(inputs, outputs)
```

The `GlobalAveragePooling2D` turns spatial feature maps into vectors, which the `Dense` layers then classify. When building a **very deep** ConvNet, add **batch normalization** and **residual connections** — the two patterns from chapter 9 that keep gradient information flowing.

Context awareness for a set of vectors

When the set is an ordered sequence, **positional encoding** adds order awareness — giving a model that handles both global context and word order, and processes long paragraphs far more effectively than RNNs or 1D ConvNets.

TransformerEncoder

Turns an input vector sequence into a **context-aware, order-aware** output sequence. If you are processing a single sequence, this is all you need.

TransformerDecoder

Takes the encoder output **and** a target sequence, and predicts what comes next in the target.

RNNs, VAEs, and diffusion models

Recurrent networks

Process sequences one step at a time, carrying state forward. Still the natural fit for **timeseries** — chapter 13 showed a small GRU beating a much larger model.

VAEs

Encode to a **distribution** rather than a point, giving a continuous, structured latent space. Good where control and interpretability matter more than fidelity.

Diffusion models

A denoising autoencoder **in a loop**, turning pure noise into an image. Behind nearly every commercial image generator today.

All

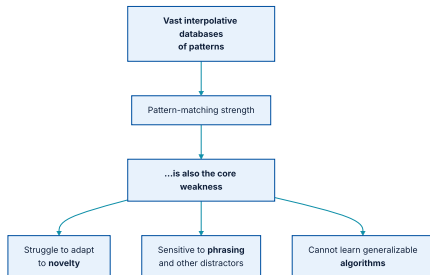
of these combine. A multimodal model splices an image encoder's output into a text sequence; a text-to-image model feeds a Transformer's embeddings to a diffusion denoiser. **The bricks fit together.**

04

Limitations, and what might lie ahead

State these plainly. They are the most useful thing you can tell a stakeholder.

Strength and weakness are the same property



Here is the catch: the mapping is often not learnable in a way that generalizes.

How to say this without hedging

- 1 **They struggle to adapt to novelty.** Parameters are fixed after training, so the model can only retrieve or replicate patterns similar to its training data. Outside that distribution, performance degrades drastically — *no matter how simple the underlying task*.
- 2 **They are sensitive to phrasing and other distractors.** Minor rewording in a prompt, or imperceptible perturbation in an image, changes the answer — which indicates the absence of robust, human-like understanding.
- 3 **They often cannot learn generalizable algorithms.** The continuous, geometric nature of these models makes them ill-suited to exact, discrete, step-by-step procedures. They approximate such processes by interpolation.

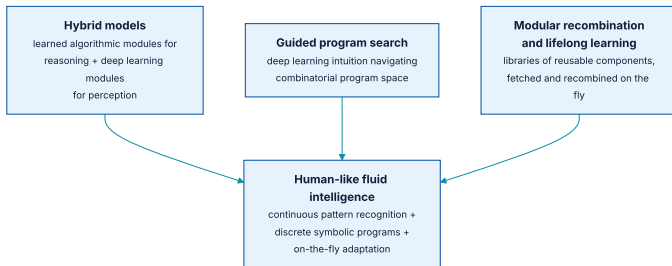
And always resist the temptation to anthropomorphize. Performance is built on **pointwise statistical patterns**, not human-like experiential grounding — which is why it is brittle at any deviation from the training data.

Five years of exponential scaling, and an open problem

By 2024 this spurred the transition to **test-time adaptation** — search or fine-tuning during inference. It yielded major breakthroughs, including o3 surpassing the human baseline on ARC-AGI-1.

But **at extreme computational cost**. Efficient, human-like adaptation remains a completely open problem, and the slightly harder **ARC-AGI-2 remains completely unsolved**. **We still need conceptual advances beyond scaling or brute-force search.**

Three developments worth watching



Deep learning excels at value-centric abstraction and lacks program-centric abstraction. Human intelligence integrates both.

The last of the three is the one with the clearest analogy to software engineering: **libraries of reusable, modular components** acquired from experience, fetched and recombined into a model adapted to the situation at hand.

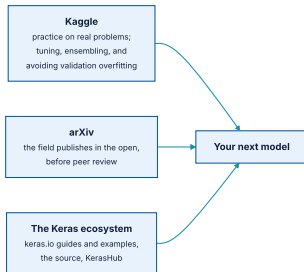
05

Staying up to date

What you learned here will not stay relevant forever.

The field moves at a frenetic pace

What you have learned in this course will not stay relevant forever, and it is not all you will need for the rest of your career.



Three free resources, and what each one is actually good for.

Kaggle: the only real way to learn is practice

Participating in a few — perhaps as part of a team — makes you familiar with the practical side of the advanced practices in chapter 18:

- ♦ **Hyperparameter tuning** at a scale where it actually matters.
- ♦ **Avoiding validation-set overfitting** when the leaderboard is watching.
- ♦ **Model ensembling**, and finding out first-hand that diversity beats individual quality.

arXiv: the field publishes in the open

Researchers upload to arXiv shortly after completion, to plant a flag without waiting months for conference acceptance. **This is what lets the field move so fast** — every new finding is immediately available to build on.

The downside: the sheer volume makes it impossible to skim, and papers are **not peer-reviewed**, so identifying what is both important and high quality is hard and getting harder. **Use Google Scholar to track publications by authors you trust.**

The Keras ecosystem

DOCS

keras.io — the main reference

GUIDES

keras.io/guides — extensive developer guides

EXAMPLES

keras.io/examples — dozens of high-quality code examples

SOURCE

github.com/keras-team/keras

HUB

github.com/keras-team/keras-hub

The examples in particular are worth reading before you write anything from scratch — most of what you need has a **maintained, tested reference implementation** there already.

Twenty chapters, six ideas

CHAPTERS	THE IDEA
1 - 3	Learning is gradient descent on a differentiable chain of geometric transformations. Everything else is engineering.
4 - 7	Generalization is the whole problem , and the workflow exists to protect it — splits, baselines, overfit-then-regularize.
8 - 12	Structure in the data buys you assumptions in the architecture. Convolution is translation invariance made into a layer.

Twenty chapters, six ideas (2 of 2)

CHAPTERS	THE IDEA
13-14	Order is a design decision. Set or sequence is the question every architecture answers differently.
15-17	Attention turns correlation into proximity , and interpolative spaces generate both generalization and hallucination.
18-20	Scale, precision, and honesty. What each lever costs, and what none of them fixes.

The through-line from chapter 5 to chapter 19 is a single claim, restated at increasing scale: **these models interpolate**. Everything they do well and everything they do badly follows from that.

Four mistakes that survive the whole course

Label leakage into validation

Random splits on temporal data; preprocessing fitted on the full dataset; `adapt()` called before the split. Every metric afterwards is fiction.

Regularizing before overfitting

You cannot tell whether a model is too small or too regularized until you have seen it **overfit once**. Get there first.

A convenient metric, not a useful one

The shortcut rule applies to your project too. Netflix never shipped the model that won its own competition.

Anthropomorphizing the model

Believing the model *understands* leads directly to deploying it where its distribution does not reach — and to being surprised.

Where to start, in your own organization

- 1 **Find a problem where the data already exists.** The workflow's first step is the one most projects skip, and data collection is what kills most of them.
- 2 **Write down the measure of success before the model.** If you cannot state it in one sentence, the project is not ready.
- 3 **Build the common-sense baseline first.** It costs an afternoon and occasionally ends the project — which is a good outcome, found early.
- 4 **Prefer a pretrained backbone.** Chapters 8, 15, and 16 all made the same point: fine-tuning beats training from scratch, decisively.
- 5 **Plan for monitoring on day one.** Distribution shift is not a research concern; it starts the day you deploy.

Go on learning, questioning, and researching

*Learning is a lifelong journey, especially in the field of AI, where we have far more unknowns on our hands than certitudes. So please go on learning, questioning, and researching. **Never stop.** Because even given the progress made so far, most of the fundamental questions in AI remain unanswered. **Many haven't even been properly asked yet.***

Chollet & Watson, Deep Learning with Python, closing words

BOOK

deeplearningwithpython.io

MODULE

[Agentic AI — cases, tech stack, and demos](#)